Ejercicio 1

Crea una array de numpy con los siguiente números primos: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29

- Muestra el último elemento. Y debes hacerlo sin saber su longitud.
- Muestra el 3º elemento
- Muestra el último y 3º elemento.
- Muestra del 2º al 5º elemento.
- Muestra el 2° y el 5° elemento
- Muestra el último y el penúltimo elemento. Y debes hacerlo sin saber su longitud.

Ejercicio 2

- Crea una array de numpy con los siguiente números primos: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 pero que el array sea de tipo "float" de 32 bits.
- Usando la función arange de numpy crea un array con los números del 100 al 200 pero sin incluir el 200. Muestra el resultado.
- Usando la función range estándar de Python crea un array con los números del 100 al 200 pero sin incluir el 200. Muestra el resultado. ¿Qué diferencia hay con arange?

Ejercicio 3

Crea la siguiente matriz:

$$\begin{pmatrix} 1 & 4 & 6 & 5 \\ 4 & 1 & 7 & 3 \\ 2 & 9 & 1 & 2 \\ 6 & 3 & 1 & 1 \end{pmatrix}$$

- Muestra el elemento de la fila 2º y la columna 3º. Es el valor del 7.
- Muestra la 3º Fila
- Muestra la 2º Columna
- Muestra la 2º y 3º Columna
- Muestra la 2º y 3º Fila
- Muestra la última columna. Debe funcionar independientemente del número de columnas.
- Muestra la 2º y 4º Columna y la 1º y 3º fila

- Muestra de la 2º a la 3º Columna y de la 1º a la 3º fila
- Muestra todas las columnas excepto la primera y la última. Debe funcionar independientemente del número de columnas.
- Muestra todas las filas excepto la primera y la última. Debe funcionar independientemente del número de filas.
- Muestra todas las columnas excepto la primera y la última y todas las filas excepto la primera y la última. Debe funcionar independientemente del número de filas y columnas.
- Imprime la matriz y haz que las cabeceras de cada columna sean A, B , C y D

Ejercicio 4

El siguiente array contiene las temperaturas medias que ha hecho en Valencia en cada mes [10.2, 10.7, 13.3, 15.8, 19.3, 23.6, 26, 25.9, 22.8, 19.1, 13.9, 10.8]

- Muestra las temperaturas cuyo valor sea mayor que 20
- Muestra las temperaturas cuyo valor sea menor que 11
- Muestra las temperaturas cuyo valor sea mayor que 20 o menor que 11

Ejercicio 5

Carga los datos de un dataset muy utilizado en la ciencia de datos con el siguiente código:

```
from sklearn.datasets import load_iris

datos=load_iris().data

resultado=load_iris().target
```

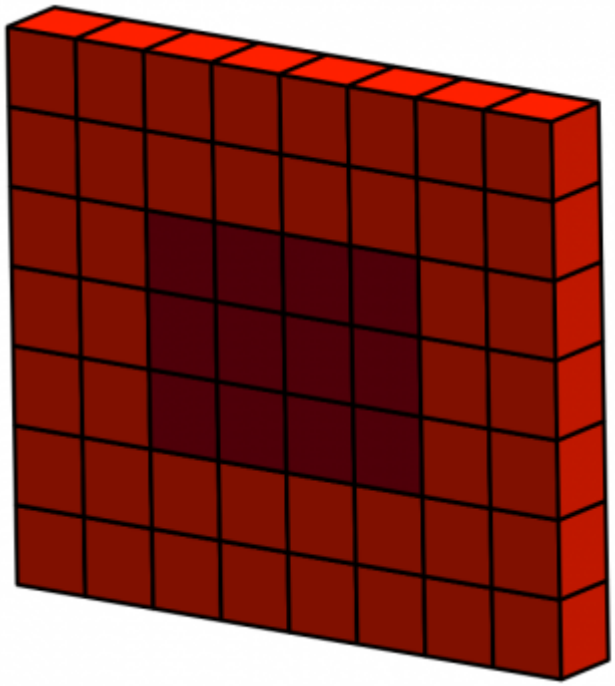
Crea un array llamado `sepal_length` con las 99 primeras filas y la 1º columna de la matriz `datos`

- Crea un array llamado `petal_length` con las 99 primeras filas y la 3º columna de la matriz `datos`
- Crea un array llamado `x` juntando las 2 columnas `sepal_length` y `petal_length`
- Crea un array llamado `y` con las 99 primeras filas del vector `resultado`

- Crea un array llamado `x` con las 99 primeras filas , la 1º columna y la 3º columna de la matriz `datos`.

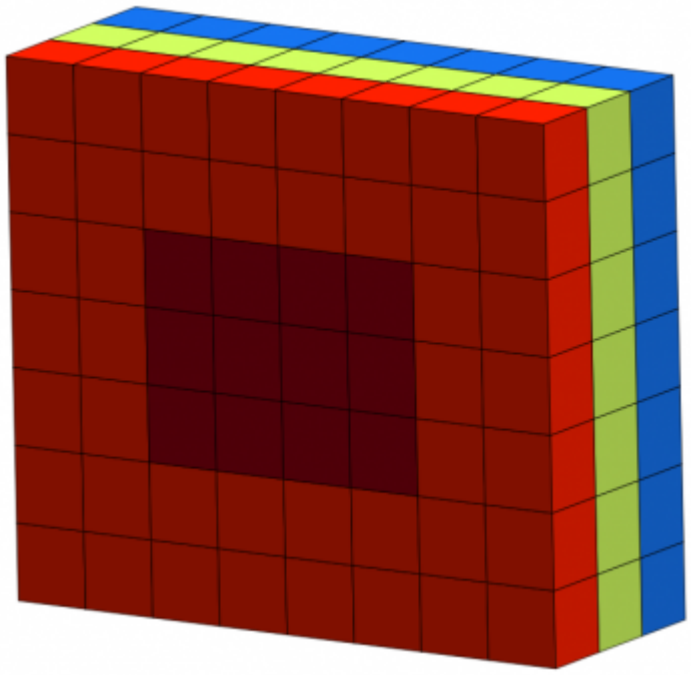
Ejercicio 6

Selecciona las celdas en rojo oscuro de la siguiente matriz:



Ejercicio 7

Selecciona las celdas en rojo oscuro de la siguiente matriz:



Ahora selecciona las celdas en rojo oscuro pero también las verdes y azules que hay por detrás de las rojo oscuro.

Ejercicio 8

Crea la siguiente matriz:

$$\begin{pmatrix} 1 & 4 & 6 & 5 \\ 4 & 1 & 7 & 3 \\ 2 & 9 & 1 & 2 \\ 6 & 3 & 1 & 1 \end{pmatrix}$$

- A todos los elementos de la matriz sumale un 10. Usando las funciones de numpy
- A todos los elementos de la matriz sumale un 10 y divídelos entre 2. Usando las funciones de numpy
- A todos los elementos de la matriz sumale un 10. Usando operadores
- A todos los elementos de la matriz sumale un 10 y divídelos entre 2. Usando operadores

Ejercicio 9

Crea la siguiente matriz:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

- Multiplica cada elemento de la matriz por sí mismo
- Multiplica la matriz por sí misma

Ejercicio 10

Crea una función llamada f que acepte como parámetro el número y que retorne el valor multiplicado por 2 y además que se le sume 1.

Crea ahora el vector de numpy [1,5,4,7,3,9,8,6] y aplícale la función f

Ejercicio 11

Crea una función llamada f que acepte como parámetro el número y que retorne lo siguiente:

- Si es valor está en el rango (-inf,3] que retorne 0
- Si es valor está en el rango [3,5] que retorne 4
- Si es valor está en el rango [5,6] que retorne 5
- Si es valor está en el rango [6,7] que retorne 6
- Si es valor está en el rango [7,9] que retorne 8
- Si es valor está en el rango [9,+inf) que retorne 10

Crea ahora el vector de numpy [1,5,4,7,3,9,8,6] y aplícale la función f

Ejercicio 12

Muestra con numpy el número de dimensiones de los siguientes arrays:

a=[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]

b=[[2, 3], [5, 7], [11, 13], [17, 19], [23, 29], [31, 37]]

c=[[[2, 3], [5, 7]], [[11, 13], [17, 19]], [[23, 29], [31, 37]]]

Ejercicio 13: Forma

Crea una función llamada mostrar_tamanyo que le pasemos un array de numpy y nos imprima el tamaño de cada una de las dimensiones.

Por ejemplo con el array [[[2, 3], [5, 7]], [[11, 13], [17, 19]], [[23, 29], [31, 37]]] deberá mostrar

El nº de elementos de la dimension 0 es 3
El nº de elementos de la dimension 1 es 2
El nº de elementos de la dimension 2 es 2

Prueba también con los siguientes arrays

a=[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]

b=[[2, 3], [5, 7], [11, 13], [17, 19], [23, 29], [31, 37]]

c=[[[2, 3], [5, 7]], [[11, 13], [17, 19]], [[23, 29], [31, 37]]]

Ejercicio 14

Muestra con numpy el tipo de datos de los siguientes arrays:

a=[2, 3, 5]

b=[2.4, 3.2, 5.6]

c=[2, 3, 5.6]

Ejercicio 15

Transforma el siguiente array en 8 bits sin signo y muestra cuánto ocupa en memoria

a=np.array([1, -2, 3])

Transforma el siguiente array en 8 bits sin signo y muestra cuánto ocupa en memoria

b=np.array([1,254,255])

Transforma el siguiente array en 8 bits con signo y muestra cuánto ocupa en memoria

b=np.array([1,127,128])

Transforma el siguiente array en 8 bits con signo y muestra cuánto ocupa en memoria

b=np.array([1,-128,-129])

Transforma el siguiente array en float32 y muestra cuánto ocupa en memoria

```
b=np.array([1,2,3])
```

Transforma el siguiente array en int32 y muestra cuánto ocupa en memoria

```
b=np.array([1.9,2.01,3.51])
```

Ejercicio 16

En ejercicios anteriores obtuviste para el problema de las flores, una matriz x y un vector y.

Une los datos en una nueva matriz de forma que cada fila de la nueva matriz contenga los datos de cada fila de la x y el dato correspondiente de la y.

Ejercicio 17

Dado el siguiente código python:

```
import matplotlib.pyplot as plt
import numpy as np
figure=plt.figure(figsize=(8,8))
axes = figure.add_subplot()
x=[-3,-2,-1,0,1,2,3]
y =  3*(1 - x)**2 * np.exp(-x**2 )  - 10*(x/5 - x**3 )*np.exp(-x**2 ) - 1./3*np.exp(-(x + 1)**2 )
axes.plot(x,y)
```

- Modifica el código que genera la variable x para que sea un array de numpy y de tipo float. Muestra la imagen

- Modifica el código que genera la variable x para que sean 10 valores entre el [-3,3]. Muestra la imagen
- Modifica el código que genera la variable x para que sean 20 valores entre el [-3,3]. Muestra la imagen
- Modifica el código que genera la variable x para que sean 40 valores entre el [-3,3]. Muestra la imagen
- Modifica el código que genera la variable x para que sean 60 valores entre el [-3,3]. Muestra la imagen
- Modifica el código que genera la variable x para que sean 100 valores entre el [-3,3]. Muestra la imagen

Ejercicio 18

Carga con numpy el fichero wine.csv que contiene una matriz de números separados por coma.

La matriz contiene una serie de columnas con características del vino y la última columna indica el tipo del vino.

- Crea una función en python llamada imprimir_datos y tendrá como parámetro una matriz de numpy de forma que:
    - Imprima cuantas características tenemos de cada tipo de vino.
    - Imprima cuantas muestra de vino tenemos
    - Imprima cuantos tipos distintos de vino tenemos y cuales son
    - Imprima los valores máximos y mínimos de cada característica
    - Retorna una nueva matriz llamada x solo con las características y un nuevo array llamado y solo con los tipos de vino
- Llama a la función imprimir_datos con los datos de wine.csv

Ejercicio 19

Tienes los datos de las ventas diarias de una tienda durante un año. Los datos incluyen ventas anormalmente altas debido a días festivos o promociones. El objetivo es limpiar los datos eliminando estos valores atípicos y analizar la tendencia general de ventas.

```
np.random.seed(0)
# Ventas diarias simuladas con media 500 y desviación estándar 50
sales_data = np.random.normal(500, 50, 365)
# Ventas atípicas por promociones
sales_data[[50, 120, 180, 250, 300]] = [1500, 1400, 1600, 1700, 1800]
```

1. Detecta y elimina los valores atípicos (aquellos que están fuera de 3 desviaciones estándar del promedio).
2. Calcula el promedio, la mediana y la desviación estándar de las ventas después de limpiar los datos.
3. Genera una vista de las ventas de cada trimestre (3 primeros meses, 3 siguientes, etc.).
4. Encuentra los 5 días con las ventas más bajas después de la limpieza de los datos.

Ejercicio 20

Tienes un conjunto de datos que contiene información demográfica de varias personas, incluyendo edad, ingresos anuales, y nivel de educación. El objetivo es realizar un análisis y encontrar patrones significativos entre estas variables.

```
np.random.seed(3)
ages = np.random.randint(18, 70, size=200)  # Edades de 200 personas
income = np.random.normal(50000, 15000, size=200)  # Ingresos anuales en un rango normal alrededor de 50,000
education_years = np.random.randint(0, 20, size=200)  # Años de educación formal
```

1. Calcula la correlación entre los años de educación y los ingresos.
2. Divide a las personas en grupos de edades (18-30, 31-45, 46-60, 61+) y calcula el ingreso promedio por grupo.
3. Encuentra el percentil 90 de ingresos y determina cuántas personas ganan por encima de ese valor.
4. Agrupa los datos en función del número de años de educación y calcula el promedio de ingresos para cada grupo.

Para calcular la correlación utiliza la función corrcoef.

Para encontrar el percentil 90 utiliza la función percentile.